COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	7/16/2026	**DATE SUBMITTED:**	7/20/2026
NAME:	Denzen D. Igloso	**INSTRUCTOR:**	Engr. Jimlord Quejado

HOA NO. _2.1_ **Arrays, Pointers and Dynamic Memory Allocation**

1. PROCEDURE OUTPUT

OUTPUT

HOA > main.cpp > Student > Student(std::string, int)

```cpp
#include <iostream>
#include <string.h>

class Student {
    private:
        std::string studentName;
        int studentAge;
    public:
        Student (std::string newName = "John Doe", int newAge = 18) {
            studentName = std::move(newName);
            std::cout << "Constructor Called." << std::endl;

        };
        ~Student(){
            std::cout << "Destructor Called." << std::endl;
        }
        Student(const Student &copyStudent){
            std::cout << "Copy Constructor Called" << std::endl;
            studentName = copyStudent.studentName;
            studentAge = copyStudent.studentAge;
        }
        void printDetails(){
            std::cout << this-> studentName << " " << this-> studentAge << std::endl;
        }
};
int main(){
    Student student1("Denzen", 25);
    Student student2(student1);
    Student student3;
    student3 = student2;

    return 0;
}
```

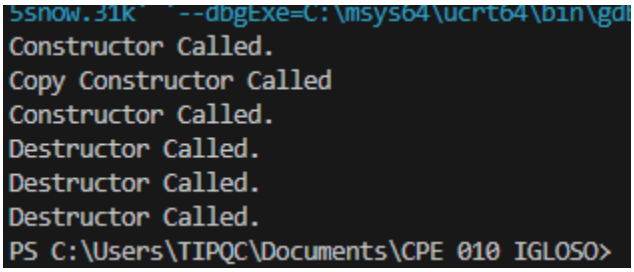
	
OBSERVATION	Based on this code, first is that the driver program executes the Constructor which are the Student (name, age) and then allocate it to the object student then move to the driver program next line which is the student2(student1), this is the copy constructor and allocate the data of student 1 to student 2 then move to the next line which is a constructor student 3 and it contains no name so it prints the john doe and to the next line of driver program, student3 = student2 this means that it copies the name and age of the student 2 to student 3 without using the copy constructor and then after this it moves back and forth to the destructor and driver program and delete one at a time the name and age of an object.

Table 2-1. Initial Driver Program

OUTPUT	HOA > main.cpp > main()
	1 #include <iostream>
	2 #include <string.h>
	3
	4 class Student {
	5 private:
	6 std::string studentName;
	7 int studentAge;
	8 public:
	9 Student (std::string newName = "John Doe", int newAge = 18) {
	10 studentName = std::move(newName);
	11 std::cout << "Constructor Called." << std::endl;
	12
	13 };
	14 ~Student(){
	15 std::cout << "Destructor Called." << std::endl;
	16 }
	17 Student(const Student& copyStudent){
	18 std::cout << "Copy Constructor Called" << std::endl;
	19 studentName = copyStudent.studentName;
	20 studentAge = copyStudent.studentAge;
	21 }
	22 void printDetails(){
	23 std::cout << this-> studentName << " " << this-> studentAge << std::endl;
	24 }
	25 };
	26 int main(){
	27 const size_t j = 5;
	28 Student studentList[j] = {};
	29 std::string namesList[j] = {"Lyrae", "Peter", "Zach", "Kenzo", "Angel"};
	30 int ageList[j] = {15, 16, 18, 19, 16};
	31
	32 return 0;
	33 }

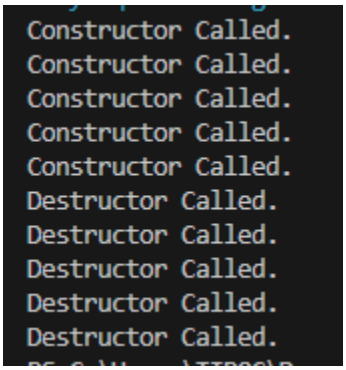
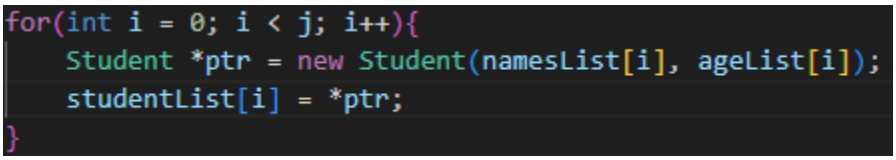
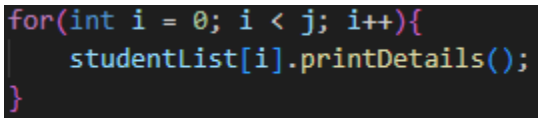
	
OBSERVATION	Based on this code, the driver program executes first which is the declaration of array size and puts it in the created constructor. After this, the Student studentList was executed 5 times using constructor and allocated the name john doe and age of 18, after this the next line of the driver program is a list of strings(name) that creates a new array that places the 5 john doe, and the next line is the a list of integer(age) that creates a new array that also places the age of the students. After this it goes back to the driver program which is the namelist[j] and delete it using destructor, after this it also removes the allocated john doe`s in an array and delete it using also the destructor.

Table 2-2. Modified Driver Program with Student Lists

LOOP A	
OBSERVATION	This loop A code, it allocated the name on the namesList[] array to the heap memory one by one on all 5 students.
LOOP B	
OBSERVATION	This loop B code, it prints the name and the age of the students by its memory from the heap memory.

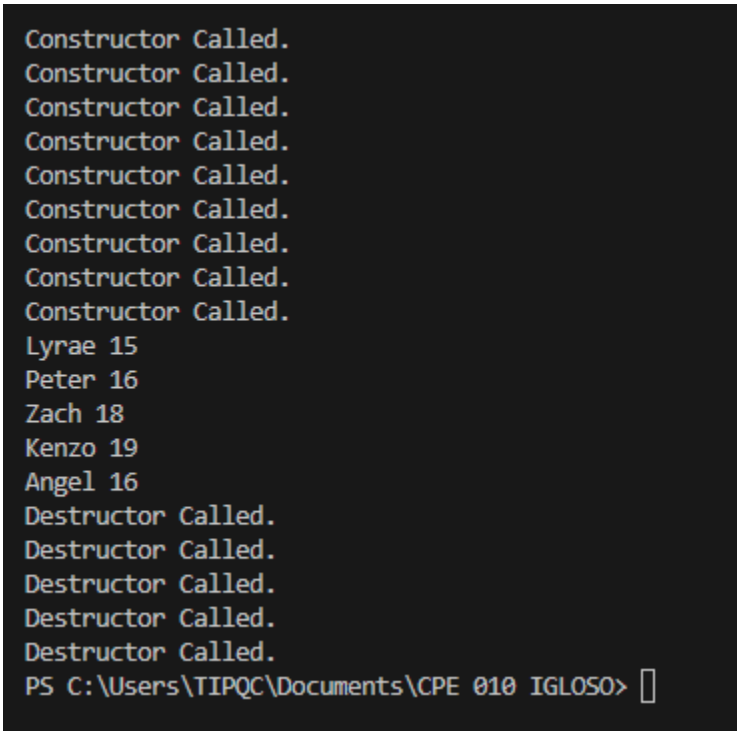
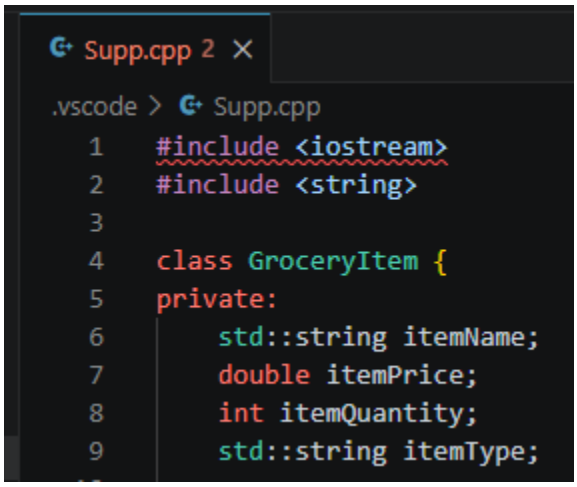
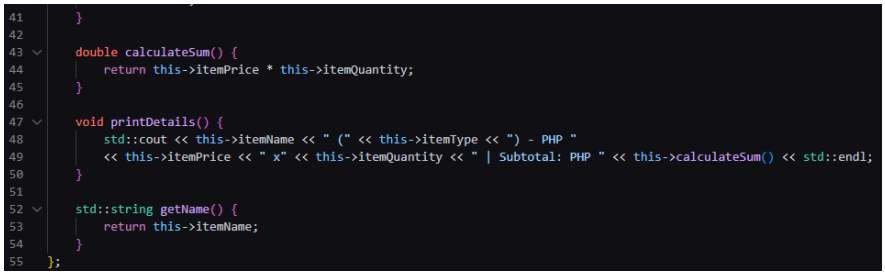
OUTPUT	
OBSERVATION	For this output, it shows the 10 constructors called. The first 5 is the allocation of memory which is the john doe and age of 18 to the array and the other 5 constructors called, after the declaration of a new array which is the namelis[] that contains the list of 5 students. This replace the 5 memory of john doe in the first array from the nameList[] which is this std::string namesList[j] = {"Lyrae", "Peter", "Zach", "Kenzo", "Angel"};. And after that it prints the memory from the heap, the names and the age of the students and after that, the 5 destructor called is the deletion of the 5 students on the heap memory.

Table 2-3. Final Driver Program

COMPUTER ENGINEERING

QUEZON CITY

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ANALYSIS	CODE
These are declarations of private member variables that define the basic attributes each grocery item object will store.	
This part sets up the public methods like the constructor, virtual destructor, copy constructor, and copy assignment operator so the program can safely create, copy, and delete grocery items.	
This section contains the regular member functions of the class that calculate the subtotal price, print out all the details of an item using the this-> pointer, and return the item's name so other parts of the program can read it.	

Supp.cpp 2

.vscode > Supp.cpp

```cpp
#include <iostream>
#include <string>

class GroceryItem {
private:
    std::string itemName;
    double itemPrice;
    int itemQuantity;
    std::string itemType;
```

```cpp
public:
    GroceryItem(std::string newName = "Unknown", int newQuantity = 0, double newPrice = 0.0, std::string newType = "Item") {
        itemName = std::move(newName);
        itemPrice = newPrice;
        itemQuantity = newQuantity;
        itemType = std::move(newType);
        std::cout << "Constructor Called." << std::endl;
    }

    virtual ~GroceryItem() {
        std::cout << "Destructor Called." << std::endl;
    }

    GroceryItem(const GroceryItem& copyItem) {
        std::cout << "Copy Constructor Called" << std::endl;
        itemName = copyItem.itemName;
        itemPrice = copyItem.itemPrice;
        itemQuantity = copyItem.itemQuantity;
        itemType = copyItem.itemType;
    }

    GroceryItem& operator=(const GroceryItem& copyItem) {
        std::cout << "Copy Assignment Operator Called" << std::endl;
        if (this != &copyItem) {
            itemName = copyItem.itemName;
            itemPrice = copyItem.itemPrice;
            itemQuantity = copyItem.itemQuantity;
            itemType = copyItem.itemType;
        }
        return *this;
    }
```

```cpp
    }

    double calculateSum() {
        return this->itemPrice * this->itemQuantity;
    }

    void printDetails() {
        std::cout << this->itemName << " (" << this->itemType << ") - PHP "
        << this->itemPrice << " x" << this->itemQuantity << " | Subtotal: PHP " << this->calculateSum() << std::endl;
    }

    std::string getName() {
        return this->itemName;
    }
};
```

This part creates the child classes Fruit and Vegetable that inherit from the GroceryItem base class, passing their unique data upward through the constructor list and overriding the destructor to properly print when they get cleaned up.

```
class Fruit : public GroceryItem {
public:
    Fruit(std::string newName, double newPrice, int newQuantity)
        : GroceryItem(newName, newQuantity, newPrice, "Fruit") {}

    ~Fruit() override {
        std::cout << "Fruit Destructor Called." << std::endl;
    }
};

class Vegetable : public GroceryItem {
public:
    Vegetable(std::string newName, double newPrice, int newQuantity)
        : GroceryItem(newName, newQuantity, newPrice, "Vegetable") {}

    ~Vegetable() override {
        std::cout << "Vegetable Destructor Called." << std::endl;
    }
};
```

This section is a global function called TotalSum that loops through the array of grocery item pointers, checks if the item hasn't been deleted (nullptr), and adds up all their subtotals to calculate the final total bill.

```
};

double TotalSum(GroceryItem* list[], int size) {
    double total = 0.0;
    for (int i = 0; i < size; i++) {
        if (list[i] != nullptr) {
            total += list[i]->calculateSum();
        }
    }
    return total;
}
```

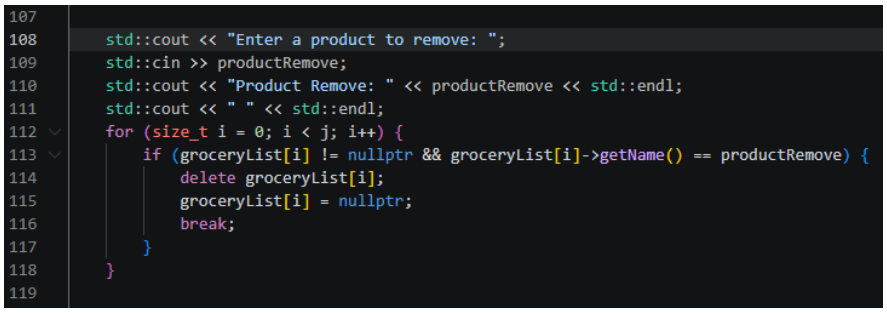
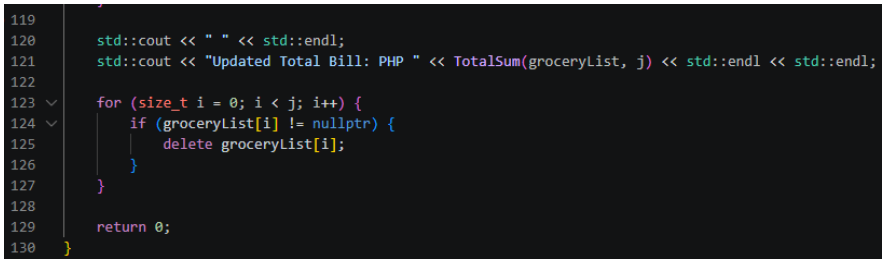
This part is the start of the main() function, where the program dynamically allocates memory for the Fruit and Vegetable objects in an array of pointers, loops through the list to print all their item details, and calculates the initial total bill.

```
int main() {
    const size_t j = 4;
    std::string productRemove;
    GroceryItem* groceryList[j] = {};

    groceryList[0] = new Fruit("Apple", 10.0, 7);
    groceryList[1] = new Fruit("Banana", 10.0, 8);
    groceryList[2] = new Vegetable("Broccoli", 60.0, 12);
    groceryList[3] = new Vegetable("Lettuce", 50.0, 10);

    std::cout << " " << std::endl;
    std::cout << "Grocery List Details --------------------------------" << std::endl;
    for (size_t i = 0; i < j; i++) {
        if (groceryList[i] != nullptr) {
            groceryList[i]->printDetails();
        }
    }

    std::cout << " " << std::endl;
    std::cout << "Total Bill: PHP " << TotalSum(groceryList, j) << std::endl << std::endl;
```

This section prints the updated total bill after removing the item, and then uses a for loop to clean up all the remaining dynamically allocated objects left in the array to fully prevent any memory leaks.	
This final section prints the updated total bill after the item is removed, loops through the list to delete all remaining dynamically allocated objects to prevent memory leaks, and successfully terminates the program.	

```
107
108     std::cout << "Enter a product to remove: ";
109     std::cin >> productRemove;
110     std::cout << "Product Remove: " << productRemove << std::endl;
111     std::cout << " " << std::endl;
112     for (size_t i = 0; i < j; i++) {
113         if (groceryList[i] != nullptr && groceryList[i]->getName() == productRemove) {
114             delete groceryList[i];
115             groceryList[i] = nullptr;
116             break;
117         }
118     }
119
```

```
119
120     std::cout << " " << std::endl;
121     std::cout << "Updated Total Bill: PHP " << TotalSum(groceryList, j) << std::endl << std::endl;
122
123     for (size_t i = 0; i < j; i++) {
124         if (groceryList[i] != nullptr) {
125             delete groceryList[i];
126         }
127     }
128
129     return 0;
130 }
```

CONCLUSION

- In this lab, I mostly focused on setting up the basic GroceryItem class with its constructors, a copy assignment operator, and a virtual destructor to handle the data, plus a simple function using this-> to print the subtotals. For the extra part, I made the Fruit and Vegetable classes inherit from the main class, and then in main(), I set up an array of pointers to hold everything, loop through them for the total bill, and added a quick cin input so the user can pick an item to delete and free up the memory. Honestly, I think I did okay since the whole thing actually runs and the user input part works, but I definitely still need to practice memory management and polymorphism because remembering where to put the virtual keyword and managing all those pointers gets pretty confusing.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini,**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, code assistance, explanation of concepts.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used gemini to correct my grammar in my analysis of the codes. I also used it to help me understand more about the concepts of copy assignments operator, the this->, virtual and help me on the ones that I got stuck with like the codes with for loop and also the codes with arrays.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.